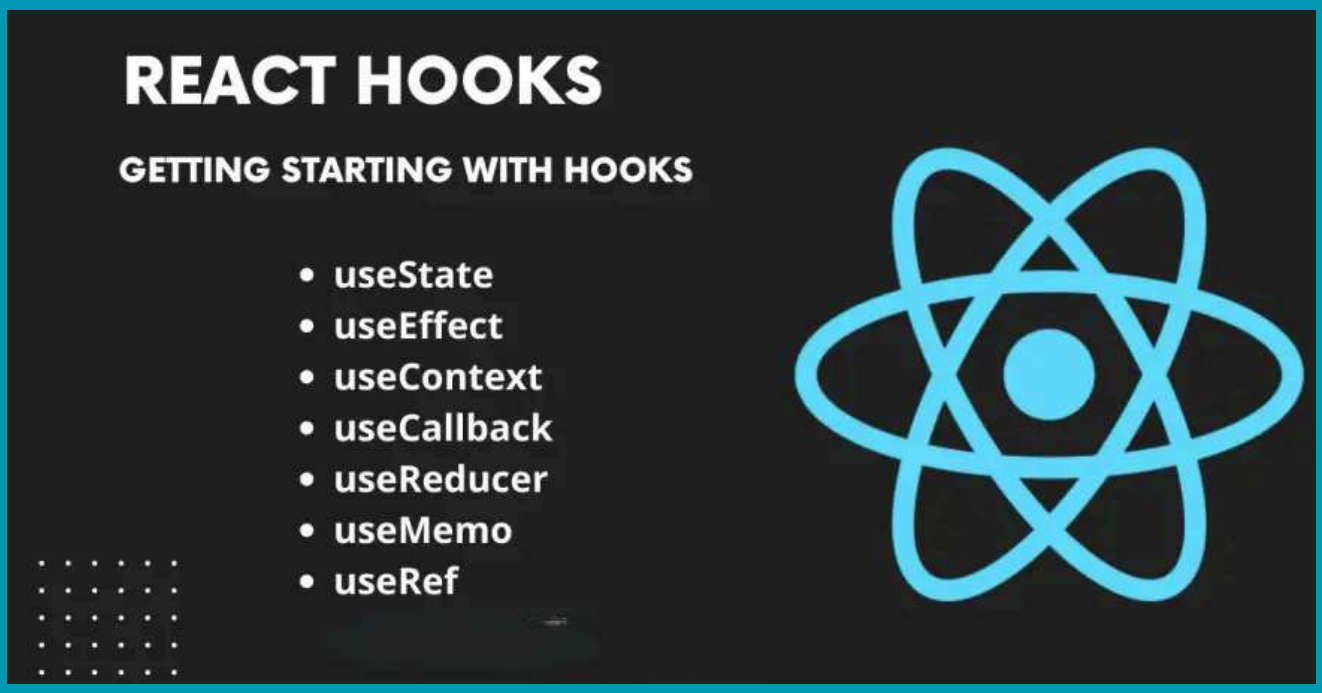

@nagma-khanam

# All 7 React Hooks in Action

My Journey in Modern
Front-End Development




@nagma-khanam

# UseState

**useState** is a **React Hook** that allows us to **add and manage state** in functional components. It returns a state variable and a function to update it, so our UI can re-render whenever the state changes. Perfect for things like counters, toggles, form inputs, and more!

It gives back **two things**:

- A **state variable** (to hold data).
- A **function** (to update that data).

```
const [counter, setCounter] = useState(0);
```

```
import React, { useState } from "react";

const App = () => {
  const [counter, setCounter] = useState(0);
  return (
    <div>
      <h1>{counter}</h1>
      <button onClick={() => setCounter((prev) => prev + 1)}>increment</button>
      <button onClick={() => setCounter((prev) => prev - 1)}>decrement</button>
    </div>
  );
};

export default App;
```

@nagma-khanam

# UseEffect

The **useEffect** hook in React is used to handle **side effects** in functional components.
Side effects include things like:

- **Fetching data** from an API
- **Updating the DOM manually**
- **Setting up subscriptions or timers**
- **Cleaning up resources** when a component unmounts

## Key Points:

- **Runs after render** – By default, it runs **after every render.**
- **Dependencies array** ([]):
  - [] → Runs only once (like **componentDidMount**).
  - [state, props] → Runs when **any of these values change.**
  - **No array** → Runs **after every render.**
- **Cleanup function:**
  - Used to clean up **timers, subscriptions,** or **listeners** when the component **unmounts** or **before re-running** the effect.

```jsx
import React, { useEffect, useState } from "react";

const App = () => {
  const [getData, setGetData] = useState(null);

  useEffect(() => {
    const URL_Todos = "https://jsonplaceholder.typicode.com/todos/1";
    const fetchAPI = async () => {
      const data = await fetch(URL_Todos);
      const jsonResponse = await data.json();
      console.log(jsonResponse);
      setGetData(jsonResponse);
    };
    fetchAPI();
  }, []);

  return;
  <div>{getData}</div>;
};

export default App;
```

@nagma-khanam

# UseRef

- The **useRef** hook in react that allow us create **mutable variables**, which will **not re-render** the component.
- **useRef** is also used for **accessing DOM Elements** and **modifying the DOM Elements**.

## Example 1: Counter using useRef

```jsx
import React, { useEffect, useRef, useState } from "react";

const App = () => {
  const [number, setNumber] = useState(0);
  const count = useRef(0);
  useEffect(() => {
    count.current = count.current + 1;
  });
  return (
    <div>
      <button onClick={() => setNumber((prev) => prev - 1)}>-1</button>
      <h2>{number}</h2>
      <button onClick={() => setNumber((prev) => prev + 1)}>+1</button>
      <h1>Render Count: {count.current}</h1>
    </div>
  );
};

export default App;
```

@nagma-khanam

## Example 2: Modifying DOM Element using useRef

```jsx
import React, { useRef } from "react";

const App = () => {
  const inputElem = useRef();
  const btnClick = () => {
    console.log(inputElem.current);
    inputElem.current.style.backgroundColor = "pink";
  };
  return (
    <div>
      <input type="text" ref={inputElem} />
      <button onClick={btnClick}>Click Here!!</button>
    </div>
  );
};

export default App;
```

@nagma-khanam

# UseMemo

- **useMemo** is a React Hook that lets you **memoize (cache) the result of a computation** so it **doesn't need to be recalculated on every render.**

- 👉 **It's useful for:**
  - **Expensive calculations** (e.g., sorting, filtering, large loops)
  - **Avoiding unnecessary re-renders** of child components by providing **stable values**

```jsx
import React, { useMemo, useState } from "react";

const App = () => {
  const [number, setNumber] = useState(0);
  const [counter, setCounter] = useState(0);
  function cubeNum(num) {
    console.log("calculation done!");
    return Math.pow(num, 3);
  }
  const result = useMemo(() => cubeNum(number), [number]);
  return (
    <div>
      <input text="number" value={number} onChange={(e) => setNumber(e.target.value)}/>
      <h2>cube of the number: {result}</h2>
      <button onClick={() => setCounter((counter) => counter + 1)}>
        counter++
      </button>
      <h2>Counter: {counter}</h2>
    </div>
  );
};
export default App;
```

# UseCallback

- **useCallback** is a **React Hook** that returns a **memoized version of a function**.
- It helps **prevent unnecessary re-creation of functions** on every render.
- This is useful when you **pass functions as props** to **child components** (to **avoid re-renders**).

```jsx
import React, { useState } from "react";
import Header from "./Components.jsx/Header";
import { useCallback } from "react";

const App = () => {
  const [count, setCount] = useState(0);
  const newFun = useCallback(() => {}, []);

  return (
    <div>
      <Header newFun={newFun} />
      <h2>{count}</h2>
      <button onClick={() => setCount((prev) =>
        prev + 1)}>Click Here!</button>
    </div>
  );
};
export default App;
```

```jsx
import React from "react";

const Header = () => {
  console.log("Header Render");
  return <div>Header</div>;
};

export default React.memo(Header);
```

@nagma-khanam

# UseContext

- **useContext** is a **React hook** that lets you **access context values** directly in **functional components**.

- It removes the need to **pass props manually** through every level of the component tree (**prop drilling**).

- **useContext** is used to **manage global data** in the **React App** when we build **small applications**.

```
import { createContext } from "react";

export const AppContext = createContext();

const ContextProvider = (props) => {
  const contactNumber = "+1 23456789";
  const name = "React Hook";
  console.log(contactNumber);
  return (
    <AppContext.Provider value={{ contactNumber, na
      {props.children}
    </AppContext.Provider>
  );
};
```

```
1  import { createRoot } from 'react-dom/client'
2  import './index.css'
3  import App from './App.jsx'
4  import ContextProvider from './context/AppContext.
5
6  createRoot(document.getElementById('root')).render(
7    <ContextProvider>
8      <App />
9    </ContextProvider>
10
11
12 )
13
```

@nagma-khanam

```jsx
import React, { useContext } from "react";
import { AppContext } from "../context/AppContext";

const Parent = () => {
  const { contactNumber } = useContext(AppContext);
  return (
    <div>
      <h2>parent</h2>
      <h3>contactNumber: {contactNumber}</h3>
    </div>
  );
};

export default Parent;
```

```jsx
import React, { useContext } from "react";
import { AppContext } from "../context/AppContext";

const SecondChild = () => {
  const { contactNumber, name } = useContext(AppContext);
  return (
    <div>
      <h2>secondchild</h2>
      <h3>contactNumber: {contactNumber}</h3>
      <h3>name: {name}</h3>
    </div>
  );
};

export default SecondChild;
```

```jsx
import React from "react";
import SecondChild from "./SecondChild";

const FirstChild = () => {
  return (
    <div>
      <h2>firstchild</h2>
      <SecondChild />
    </div>
  );
};

export default FirstChild;
```

```jsx
import FirstChild from "./Components.jsx/FirstChild";
import Parent from "./Components.jsx/Parent";

const App = () => {
  return (
    <div>
      <Parent />
      <FirstChild />
    </div>
  );
};
export default App;
```

@nagma-khanam

# useReducer

- useReducer is an alternative to useState when state logic is complex (e.g., multiple values, or updates depending on previous state).

- It works like Redux but inside a single component.

const [state, dispatch] = useReducer(reducer, initialState)

- **state** → current state
- **dispatch** → function to send **actions**
- **reducer** → function that decides how **state changes**
- **initialState** → starting state

```jsx
import React, { useState } from "react";
import { useReducer } from "react";
const App = () => {
  const initialState = { count: 0 };
  const reducer = (state, action) => {
    switch (action.type) {
      case "increment": {
        return { count: state.count + 1 };
      }
      case "decrement": {
        return { count: state.count - 1 };
      }
      default: {
        return state;
      }
    }
  };
  const [state, dispatch] = useReducer(reducer, initialState);
  return (
    <div>
      <h1>{state.count}</h1>
      <button onClick={() => dispatch({ type: "increment" })}>Increment</button>
      <button onClick={() => dispatch({ type: "decrement" })}>Decrement</button>
    </div>
  );
};

export default App;
```

Which React hook do you use most often for handling state in your components?

💬 Share your go-to use case in the comments!

For more dev insights, follow

***@nagma-khanam***